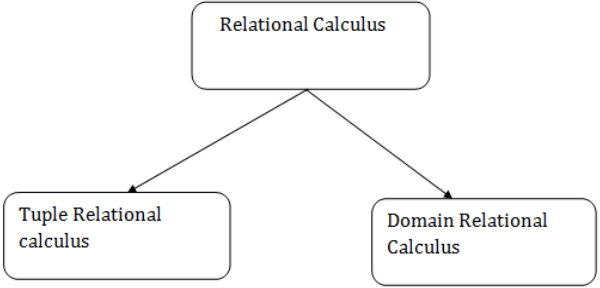
Relational Calculus in DBMS

Relational Calculus

- Relational calculus is a non-procedural query language. In the non-procedural query language, the user is concerned with the details of how to obtain the end results.
- The relational calculus tells what to do but never explains how to do.

Types of Relational calculus:



Why it is called Relational Calculus?

It is based on Predicate calculus, a name derived from branch of symbolic language. A predicate is a truth-valued function with arguments. On substituting values for the arguments, the function result in an expression called a proposition. It can be either true or false. It is a tailored version of a subset of the Predicate Calculus to communicate with the relational database.

Many of the calculus expressions involves the use of Quantifiers. There are two types of quantifiers most important:

Universal Quantifiers: The universal quantifier denoted by $\forall$ is read as for all which means that in a given set of tuples exactly all tuples satisfy a given condition.

Existential Quantifiers: The existential quantifier denoted by $\exists$ is read as for all which means that in a given set of tuples there is at least one occurrences whose value satisfy a given condition.

Free and Bound Variables:

A tuple variable t is bound if it is quantified which means that if it appears in any occurrences, a variable that is not bound is said to be free.

Free and bound variables may be compared with **global and local variable of programming languages.**

1. Tuple Relational Calculus (TRC)

- The tuple relational calculus is specified to select the tuples in a relation. In TRC, filtering variable uses the tuples of a relation.
- The result of the relation can have one or more tuples.

Notation:

```
{T | P (T)}   or {T | Condition (T)}
```

Where

T is the resulting tuples

P(T) is the condition used to fetch T.

```
First_Name      Last_Name     Age
----------      ---------     ----
Ajeet           Singh         30
Chaitanya       Singh         31
Rajeev          Bhatia        27
Carl            Pratap        28
```

Lets write relational calculus queries.

Query to display the last name of those students where age is greater than 30

```
{ t.Last_Name | Student(t) AND t.age > 30 }
```

In the above query you can see two parts separated by | symbol. The second part is where we define the condition and in the first part we specify the fields which we want to display for the selected tuples.

The result of the above query would be:

```
Last_Name
---------
Singh
```

Query to display all the details of students where Last name is 'Singh'

```
{ t | Student(t) AND t.Last_Name = 'Singh' }
```

Output:

```
First_Name      Last_Name      Age
----------      ---------      ----
Ajeet           Singh          30
Chaitanya       Singh          31
```

2. Domain Relational Calculus (DRC)

- The second form of relation is known as Domain relational calculus. In domain relational calculus, filtering variable uses the domain of attributes.
- Domain relational calculus uses the same operators as tuple calculus. It uses logical connectives ∧ (and), ∨ (or) and ┐ (not).
- It uses Existential (∃) and Universal Quantifiers (∀) to bind the variable.

Notation:

```
{ a1, a2, a3, ..., an | P (a1, a2, a3, ... ,an)}
```

Where

a1, a2 are attributes
P stands for formula built by inner attributes

For example:

```
{< article, page, subject > |  ∈ javatpoint ∧ subject = 'database'}
```

Output: This query will yield the article, page, and subject from the relational javatpoint, where the subject is a database.

```
First_Name      Last_Name     Age
----------      ---------     ----
Ajeet           Singh         30
Chaitanya       Singh         31
Rajeev          Bhatia        27
Carl            Pratap        28
```

Query to find the first name and age of students where student age is greater than 27

```
{< First_Name, Age > | ∈ Student ∧ Age > 27}
```

Note:

The symbols used for logical operators are: ∧ for AND, ∨ for OR and ┐ for NOT.

Output:

```
First_Name      Age
----------      ----
Ajeet           30
Chaitanya       31
Carl            28
```